

Direct Addressing — Exercises 11.1

Exercise 11.1-1

Suppose that a dynamic set S is represented by a direct-address table T of length m . Describe a procedure that finds the maximum element of S . What is the worst-case performance?

Solution

Since each key k corresponds directly to slot $T[k]$:

$$T[k] = \begin{cases} \text{pointer to element} & \text{if } k \in S \\ \text{NIL} & \text{otherwise} \end{cases}$$

To find the maximum element, scan from index $m - 1$ down to 0 and return the first non-NIL entry.

Algorithm 1 MAXIMUM(T, m)

```
for  $i \leftarrow m - 1$  downto 0 do
    if  $T[i] \neq \text{NIL}$  then
        return  $i$ 
    end if
end for
return NIL
```

Worst-case (when S is empty or maximum is near index 0):

$$\boxed{O(m)}$$

Exercise 11.1-2

Describe how to use a bit vector of length m to represent a dynamic set of distinct elements with no satellite data. Dictionary operations should run in $O(1)$ time.

Solution

Use a bit vector $V[0 \dots m - 1]$:

$$V[k] = \begin{cases} 1 & \text{if } k \in S \\ 0 & \text{otherwise} \end{cases}$$

Example (set $\{2, 4, 6, 10, 16\}$ in length 20):

00101010001000001000

Algorithm 2 BITMAP-SEARCH(V, k)

```
if  $V[k] = 1$  then
    return  $k$ 
else
    return NIL
end if
```

Algorithm 3 BITMAP-INSERT(V, k)

```
 $V[k] \leftarrow 1$ 
```

Algorithm 4 BITMAP-DELETE(V, k)

```
 $V[k] \leftarrow 0$ 
```

Each operation runs in:

$$O(1)$$

Exercise 11.1-3

Suggest how to implement a direct-address table in which keys need not be distinct and elements can have satellite data. All operations should run in $O(1)$ time.

Solution

Use a direct-address table where each slot points to a doubly linked list.

$$T[k] \rightarrow \text{list of elements with key } k$$

Operations:

INSERT: Append element to head of list in $O(1)$ time.

DELETE: Given pointer to element, remove from doubly linked list in $O(1)$ time.

SEARCH: Return head of list in $O(1)$ time.

Thus all operations run in:

$$O(1)$$

Exercise 11.1-4

★ Describe how to implement direct addressing on a huge array without initializing the entire array. All operations and initialization must run in $O(1)$ time.

Solution

Use two arrays:

- Huge array A
- Auxiliary stack S

Initially:

$$S = \emptyset$$

Each stored object contains:

- Key
- Pointer to its index in S

Each entry in S contains:

- Key
- Pointer back to A

INSERT(x):

- Push key onto stack S
- Store pointer in $A[x]$

SEARCH(x):

- Check $A[x]$
- Verify pointer consistency with S

DELETE(x):

- Remove element from stack S
- Fill hole with last stack element
- Update pointer in A

All operations take:

$$\boxed{O(1)}$$

Initialization also takes $O(1)$ since A is never explicitly initialized.